

RetireSafe

The Beginner's Guide

Everything you need to understand the project from zero: how the internet finds things, how cloud resources get their names, how a deleted resource can be taken over, how email trust works, what Terraform and access logs are, the statistics we use, what our research found, and how RetireSafe fits it all together.

If you are...	Read these first
New to networking	Part B (fundamentals), then the glossary at the end
Comfortable with DNS / cloud	Part C (core concepts) and Part D (the maths)
Preparing the pitch	§1, §10–12, Part E (our findings), the FAQ
Writing code for the demo	§12 (architecture), §21 (running the code)

No prior security knowledge is assumed. Every technical term is explained when it first appears and again in the glossary. Numbers quoted from our own research come from real data and are reproducible from the project repository.

Contents

1. The project in one page	4
2. A story: the event website	5
3. How the internet finds things: DNS	6
Names are hierarchical	6
DNS record types you need	6
How a lookup works	6
NXDOMAIN, TTL and friends	6
Registrable domain and the Public Suffix List	6
4. Cloud resources and their names	7
What “released” and “reassignable” mean	7
How we check whether an S3 bucket exists	7
New protection: account regional namespaces (2026)	7
5. Subdomain takeover and dangling references	8
Why this is so powerful for an attacker	8
References are not only DNS	8
6. Email trust: SPF in five minutes	9
7. Infrastructure as Code: Terraform basics	9
8. Web access logs	10
Words we use about traffic	10
9. Protections that already exist	10
10. The five conditions for a takeover	11
11. The evidence ladder and the four verdicts	11
12. How RetireSafe works (architecture in plain words)	12
What a blocked result could look like	12
The fingerprint catalogue	12
13. Rates, silence and the Poisson idea	13
Rate	13
The Poisson model	13
The quarantine window D^*	13
14. Uncertainty and confidence intervals	14
Wilson interval (for proportions)	14
15. Base rates: why we confirm before we alarm	14
16. Calibration: can we trust the probabilities?	15
How we test it: hold-out validation	15
Reliability diagram	15
Scores	15
17. Blast radius and the risk score	15

18. Our three test beds in plain words	16
TB1: How common are hijackable leftovers?	16
TB2: How long is long enough?	16
TB3: How much hidden dependence is behind an “idle” resource?	16
19. The real-world incidents: cheat sheet	17
How to talk about the evidence honestly	17
20. Ethics and responsible disclosure	18
21. Running the project code	18
22. FAQ: questions judges may ask	19
23. Glossary	20
24. Learning path and self-check	21
Read in this order (about 3 hours)	21
Can you explain these? (self-check)	21

1. The project in one page

The problem. Companies constantly create and delete things in the cloud: storage buckets, web apps, temporary event websites. Deleting the *thing* is easy. But other systems still hold the thing's *address*: a DNS record, a link in some code, a line in an email-security setting. If that address can be re-registered by a stranger after the deletion, the stranger inherits all the trust and traffic that still flows to it, often under the original company's own domain name.

The user. A platform or cloud engineer who is about to delete or migrate a resource, supported by the security team.

The decision we help with. *What must be removed, migrated or protected before this resource can safely be released?*

What RetireSafe does. Before a deletion happens, it gathers evidence from every place that trust can hide (DNS, code, infrastructure files, email settings, traffic logs), checks whether anyone else could re-claim the name, and gives a clear verdict with reasons: **cleared**, **blocked** (plus a fix), **retain the name**, or **unknown** (plus what evidence is missing). It never just says “safe”.

Key idea

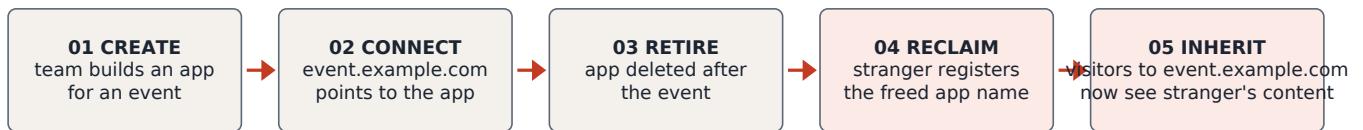
Deleting infrastructure does not automatically delete the trust placed in its address. RetireSafe makes retirement an evidence-based decision instead of a guess.

Why it matters (real incidents). Attackers have taken over abandoned addresses belonging to the US CDC, a French government Olympics website, big consulting firms and universities. One peer-reviewed study (NSDI 2024) counted 20,904 hijacked cloud resources. These cases are summarised in §19.

One-sentence pitch. *“RetireSafe is a pre-flight check for deleting cloud resources: it finds the references that would survive the deletion, tells you whether someone could hijack them, and blocks the deletion with a precise fix until it is safe.”*

2. A story: the event website

This invented example (it is *not* a real incident) shows the whole problem. Keep it in mind; every later concept maps back onto it.



Step by step:

- **Create.** The team creates a cloud web app. The cloud provider gives it a name like event-app.azurewebsites.net.
- **Connect.** To get a nice address, they add a DNS record saying event.example.com is an alias of event-app.azurewebsites.net. Posters, emails and partner websites start linking to event.example.com.
- **Retire.** After the event, someone deletes the app to save money. Nobody deletes the DNS alias, because a different team manages DNS.
- **Reclaim.** The name event-app is now free in the provider's system. An attacker creates a new app with exactly that name in *their own* account.
- **Inherit.** The alias still points to that name, so everyone visiting event.example.com now gets the attacker's site, under example.com's trusted name.

Watch out

The company still owns example.com. What it lost is control of the thing event.example.com points to. That gap between the trusted name and the resource is the vulnerability.

The same pattern happens without any custom domain: an old app or script can contain a direct URL to a storage bucket that was deleted. If the bucket name is re-created by a stranger, the old app downloads the stranger's files.

3. How the internet finds things: DNS

Computers talk to each other using numeric **IP addresses** (like 93.184.215.14). Humans use names (like example.com). The **Domain Name System (DNS)** is the internet's phone book that turns names into addresses.

Names are hierarchical

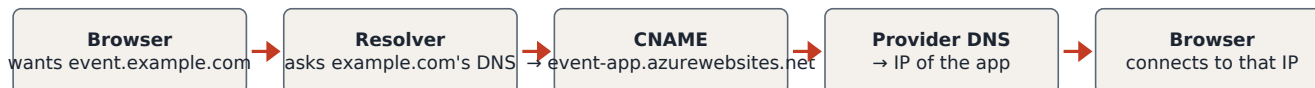
Read a name right to left: shop.event.example.com → top-level domain (TLD) com → domain example.com → subdomain event.example.com → sub-subdomain shop.event.example.com. Whoever controls example.com's DNS can create any number of subdomains for free.

DNS record types you need

Type	What it says	Example
A / AAAA	This name lives at this IPv4 / IPv6 address	example.com → 93.184.215.14
CNAME	This name is an alias of another name; go look that one up instead	event.example.com → event-app.azurewebsites.net
NS	Which servers are in charge of this domain's records	example.com → ns1.dnshost.net
TXT	Free text; used for SPF email rules and ownership proofs	v=spf1 include:_spf.mailer.com -all

How a lookup works

Your computer asks a **resolver** (usually run by your ISP, company or a service like Cisco Umbrella). The resolver asks the **authoritative** servers for the domain and follows any aliases until it reaches an address:



Following alias after alias is called walking the **CNAME chain**. Our first test bed (TB1) does exactly this for 16,000 real names.

NXDOMAIN, TTL and friends

Term	Meaning
NXDOMAIN	"Non-existent domain": the DNS answer for a name that does not exist. A CNAME whose target returns NXDOMAIN is a dangling (broken) alias.
TTL	Time-to-live; how many seconds resolvers may cache an answer. Old answers can linger for that long after you change a record.
Registrar / registry	You rent a domain like example.com from a registrar for a period. If you stop paying, it expires and anyone can register it.
Zone / zone export	All the DNS records of a domain. A zone export is a file listing them; RetireSafe reads it to find references.

Registrable domain and the Public Suffix List

Some suffixes are shared by many owners: co.uk, github.io, s3.amazonaws.com. The **Public Suffix List** records these. The part a single owner can actually register is the **registrable domain** (also called eTLD+1): for a.b.example.co.uk it is example.co.uk. We use it to ask "is the *whole domain* unregistered?" (the worst case, because anyone can buy it).

Key idea

DNS records are just pointers. Nothing in DNS checks that the thing being pointed at still belongs to you.

4. Cloud resources and their names

A **cloud resource** is anything you rent from a cloud provider: a storage bucket, a web app, a load balancer, a database. Many of them get a **public name** inside the provider's own domain, and that name is what DNS aliases and code point to.

Service	Name looks like	Who can own that name
AWS S3 bucket (storage)	my-bucket.s3.amazonaws.com	Historically: any AWS account, first come first served, across all of AWS (a global namespace)
AWS S3 static website	my-bucket.s3-website-us-east-1.amazonaws.com	Same. For a custom domain, the bucket name must equal the hostname, e.g. bucket.assets.example.com
Azure App Service	my-app.azurewebsites.net	Any Azure customer, while the name is free
AWS Elastic Beanstalk	my-env.us-east-1.elasticbeanstalk.com	Any AWS customer, while the name is free
GitHub Pages	user.github.io + custom domain	Protected if you verify the domain

What “released” and “reassignable” mean

When you delete a resource, its name is **released**. If the provider lets someone else create a resource with the same name, the name is **reassignable** (or **reclaimable**). That is the attacker's entry point. Some names are reserved for a while or protected by verification; others become available to anyone.

How we check whether an S3 bucket exists

We ask the S3 web API for the bucket and read the answer. This is read-only: it does not create anything.

S3 answer (HTTP status)	Meaning for us
404 with error code NoSuchBucket	The bucket does not exist. In the global namespace anyone could create it, so it is reclaimable
403 Forbidden	It exists but is private (someone owns it)
301 Moved Permanently	It exists, in a different AWS region
200 OK	It exists and is publicly readable

New protection: account regional namespaces (2026)

According to AWS (April 2026), buckets can now be created in an **account regional namespace** that other AWS accounts cannot reclaim. This removes the takeover risk for *new* buckets in that namespace. Older buckets in the global namespace remain at risk and cannot simply be renamed, so they need careful migration. RetireSafe must recognise which namespace a bucket is in.

Watch out

“The bucket is gone” and “the name is safe” are different statements. A deleted global-namespaces bucket is exactly what an attacker wants.

5. Subdomain takeover and dangling references

Dangling reference: any pointer (DNS record, URL in code, email rule, config) that still points at a resource you no longer control. **Subdomain takeover:** an attacker claims the resource behind a dangling DNS record and so controls what is served on your subdomain.

Why this is so powerful for an attacker

- **Borrowed reputation.** Content appears under something.yourcompany.com, which people, spam filters and search engines trust.
- **Phishing and scams.** Fake login pages and fake CAPTCHAs on a real company domain. In the CDC case, visitors were sent to scams and scareware.
- **Valid HTTPS.** Certificate authorities that use domain validation issue a certificate to whoever can serve a challenge file on that hostname. The hijacker controls that content, so the browser can show the padlock.
- **Cookies.** Cookies a site sets for the whole parent domain are also sent to every subdomain, so a hijacked subdomain may receive visitors' session cookies. The NSDI study reported stolen cookies among the abuses.
- **Search-engine spam (SEO).** In the NSDI study, 75% of the observed abuse was blackhat SEO: using the trusted domain to push spam pages up in search results.
- **Malware delivery.** If software downloads scripts, updates or packages from the reclaimed address, it may fetch the attacker's files.

References are not only DNS

Where	Dangling reference looks like
DNS	CNAME to a deleted app or bucket
Web page / JS code	<script src="https://old-assets.s3.amazonaws.com/lib.js">
Package / update config	an APT source or download URL pointing at a deleted bucket
Email (SPF)	include: of a domain that has expired (see §6)
Identity / validation	an old WHOIS or validation server name that clients still use

watchTowr's 2025 research re-registered about 150 abandoned S3 buckets and received more than 8 million requests in two months, including for a file whose documentation link had been removed in 2015. **Fixing the documentation did not stop the clients.**

Watch out

Integrity checks can still save you. If the consumer verifies a signature, a hash (Subresource Integrity), or the bucket's owner account, the attacker's replacement is rejected even if the name was reclaimed. That is condition 5 in §10.

6. Email trust: SPF in five minutes

Anyone can write any “From” address on an email, so receivers check whether the sending server is authorised. **SPF (Sender Policy Framework)** is a TXT record in DNS listing which servers may send mail for a domain:

```
example.com.  TXT  "v=spf1 include:_spf.mailer-co.com ip4:203.0.113.5 -all"

v=spf1          this is an SPF record
include:_spf.mailer-co.com  also trust every server mailer-co.com lists
ip4:203.0.113.5    trust this one address
-all              reject everything else
```

The danger is in **include:**. It hands part of your authority to *another* domain. If that other domain expires and an attacker registers it, the attacker can list their own servers, and mail they send “from” you can pass SPF.

Example

Guardio's SubdoMailing research (2024) found exactly this: an SPF rule at Swatch still included a long-abandoned domain that had been re-registered, and an MSN subdomain aliased to a domain re-registered in 2022. Spoofed email passed SPF. Guardio (vendor figure) reported more than 8,000 affected domains.

SPF is one of three related checks. **DKIM** adds a cryptographic signature to messages, and **DMARC** tells receivers what to do when checks fail and requires them to align with the visible From domain. So an SPF pass alone does not defeat every check, but it helps an attacker a lot. SPF also has a limit of 10 DNS lookups per evaluation.

Key idea

Retiring a domain or a mail service must include cleaning up every SPF include: that points at it. This is on RetireSafe's roadmap, not in the first demo.

7. Infrastructure as Code: Terraform basics

Modern teams describe their cloud resources in text files instead of clicking in a console. **Terraform** is the most common tool. You write what you want; Terraform works out what to create, change or delete.

```
resource "aws_s3_bucket" "event_assets" {
  bucket = "event-assets-2026"

  lifecycle {
    prevent_destroy = true  # refuse any plan that would delete this
  }
}
```

Concept	Meaning
state	Terraform's record of which real resources it manages
terraform plan	Shows what would change: create (+), update (~), destroy (–)
terraform apply / destroy	Makes the changes / deletes resources
terraform show -json	Machine-readable plan. RetireSafe reads this to learn what is about to be deleted
prevent_destroy	Guard that blocks destructive plans <i>while configured</i> . If someone deletes the whole resource block, the guard goes with it, and it knows nothing about outside consumers

8. Web access logs

Every time a web server answers a request it can write one line to an **access log**. Logs are how we learn whether anyone is still using a resource. Here is a real line from the NASA 1995 log we used:

```
199.72.81.55 - - [01/Jul/1995:00:00:01 -0400] "GET /history/apollo/ HTTP/1.0" 200 6245
```

Piece	Value	Meaning
client	199.72.81.55	Who asked (an IP address or hostname)
ident, user	- -	Usually empty
timestamp	01/Jul/1995:00:00:01 -0400	When (with time-zone offset)
request	GET /history/apollo/ HTTP/1.0	Method, resource path , protocol
status	200	HTTP result: 200 OK, 304 not modified, 404 not found...
bytes	6245	Size of the response

Words we use about traffic

Term	Meaning
resource	One URL path, e.g. /shuttle/countdown/count.gif
consumer / client	A person or program that requests a resource
distinct clients	How many different requesters, not how many requests
/24 network	IPv4 addresses sharing their first three numbers (e.g. 199.72.81.x). A rough proxy for “same organisation or ISP block”
external client	A client from outside the organisation (here: not ending in nasa.gov). The deleting team usually cannot contact them
recency window	The last few days of activity, which is what a dashboard shows

Watch out

Request counts are not victim counts. 8 million requests to a reclaimed bucket prove that a dependency survives; they do not prove 8 million infections.

9. Protections that already exist

This problem is well known, and providers have built defences. RetireSafe's job is to **recognise** them, not ignore them, so it does not raise false alarms.

Protection	What it does	Limit
S3 account regional namespace	Other accounts cannot reclaim those bucket names	Only for buckets created there; legacy buckets need migration
S3 bucket owner condition	Requests can say “only if owned by account X”; a stranger's bucket is rejected	Only helps if the <i>consumer</i> sends it
Azure protections	Dangling-DNS detection, alias records, domain verification	Must be configured
GitHub Pages verification	Verified custom domains are protected from takeover	Within GitHub's documented scope
Terraform prevent_destroy	Blocks deletion plans	Lost if the resource block is removed; no knowledge of consumers

Key idea

The innovation is not discovering subdomain takeover (that's known). It is coordinating a safe retirement across DNS, code, infrastructure, traffic and these protections, with evidence.

10. The five conditions for a takeover

A broken link alone is **not** a vulnerability. A takeover needs **all five** of these to be true at once. If we can prove even one is false, that path is safe.

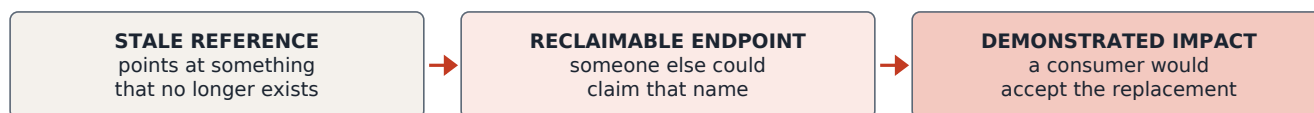
#	Condition	Question to ask	In the event-website story
1	Resource released	Will the resource be deleted?	Yes, the app is deleted
2	Name reassignable	Can someone else then create a resource with that name?	Yes, azurewebsites.net names are first come, first served
3	Reference survives	Does a DNS record, code link or rule still point there?	Yes, the CNAME was never removed
4	Consumer remains	Does anyone or anything still use the reference?	Yes, posters and partner sites still link to it
5	Controls permit impact	Would the consumer accept a stranger's content?	Yes, browsers just load whatever is served

takeover possible = C1 AND C2 AND C3 AND C4 AND C5

This turns a vague worry into a checklist. To make a deletion safe you only need to break **one** condition, and RetireSafe picks the cheapest or safest one: remove the reference (C3), keep the name (C1), move to a protected namespace (C2), or add an owner/integrity check (C5).

11. The evidence ladder and the four verdicts

Not every finding is equally serious. We place each one on a ladder and never claim a rung we have not shown:



Verdict	When	What the engineer gets
CLEARED	At least one condition is provably false for every surviving reference	An evidence record saying what was checked
BLOCKED	Every condition that could be checked is true	The exact reference(s), the reason, and a fix (patch)
RETAIN NAME	People or software still depend on it	Advice to keep owning the name (a placeholder) instead of releasing it
UNKNOWN	Some condition could not be evaluated	Which evidence is missing and how to get it

Key idea

Saying “unknown” honestly is a feature. A tool that always answers “safe” or “unsafe” will be wrong in ways nobody notices.

12. How RetireSafe works (architecture in plain words)

Think of an airline pre-flight checklist: before take-off, several independent checks must pass, and any failure stops the plane with a specific reason. RetireSafe does the same before a deletion.



Stage	Inputs	What it does	Built on
① Collect	Terraform plan, DNS zone export, code repository, SPF records	Lists every place that points to the resource being deleted	new code
② Probe	Fingerprint catalogue, live checks	Asks: does the target still exist? Could someone else claim it? Is a protection switched on?	TB1
③ Observe	Access / CDN logs	Measures how often it's used, how long a silence proves it's dead, how many outsiders depend on it	TB2, TB3
④ Fuse	Everything above	Builds a graph: Resource → Name → Reference → Consumer, each link with its evidence	new code
⑤ Decide	The graph	Evaluates the five conditions; scores risk; picks a verdict	maths in Part D
⑥ Act	The verdict	Blocks the deletion with a reason, proposes a patch, writes an evidence record	new code

What a blocked result could look like

```
BLOCKED aws_s3_bucket.event_assets (legacy global namespace -> reclaimable)
<- dns assets.event.example.com CNAME event-assets.s3.amazonaws.com [zone.txt:42]
<- code static/js/app.js:118 "https://event-assets.s3.amazonaws.com/lib.js"
consumers: 1,204 clients in 30 days, 97% external, last seen 2 days ago
fix: remove CNAME, rewrite URL, keep bucket (prevent_destroy) for 63 more days
```

This output is an illustration of the planned format, not a real scan.

The fingerprint catalogue

To know which provider a CNAME points to and how that provider behaves after deletion, we use the community catalogue **can-i-take-over-xyz**. It lists 76 services with the name pattern to match (e.g. azurewebsites.net), the tell-tale sign of an unclaimed resource (NXDOMAIN or a specific error page), and whether takeover is possible (36 vulnerable, 26 not vulnerable, 14 edge cases).

13. Rates, silence and the Poisson idea

The hardest question in retirement is: *“Nobody has used this for a while. Is it really dead, or just quiet?”* Statistics lets us answer with a number instead of a feeling.

Rate

The **rate** λ (lambda) is the average number of requests per day. If a resource got $n = 30$ requests in $T = 60$ days, our best estimate is $\lambda = n / T = \mathbf{0.5 \text{ per day}}$. (This simple estimate is called the maximum-likelihood estimate.)

The Poisson model

A **Poisson process** is the simplest model of random events that happen independently at a steady average rate, like raindrops on a window. Under that model, the probability of seeing **zero** events in a window of D days is:

$$P(\text{no requests in } D \text{ days}) = e^{(-\lambda \cdot D)} \quad (e \approx 2.718)$$

Example

A resource with $\lambda = 0.5$ requests/day stays silent for 10 days. $P = e^{(-0.5 \times 10)} = e^{-5} \approx 0.0067$, about 0.7%. So 10 days of silence is strong evidence that its users are really gone. For a resource with $\lambda = 0.05/\text{day}$, $e^{(-0.5)} \approx 0.61$: a 61% chance of 10 silent days even if it is alive, so that silence proves almost nothing.

The quarantine window D^*

Turn it around: how long must we wait so that silence would only happen by chance 1% of the time ($\alpha = 0.01$)? Solve $e^{(-\lambda D)} = \alpha$:

$$D^* = -\ln(\alpha) / \lambda = 4.605 / \lambda \quad (\text{for } \alpha = 1\%)$$

Rate λ	Means	D^* (99% sure it's dead)
10 / day	busy	0.46 days ($\approx$ 11 hours)
1 / day	daily	4.6 days
0.25 / day	about weekly-ish	18 days
0.1 / day	every 10 days	46 days
1 / 30 days	monthly	138 days

Key idea

Rare resources need long quarantines. One global rule such as “delete after 30 idle days” is far too short for anything used about once a month.

Watch out

You can't measure what your window can't see. If your log covers 27 days, a yearly job is invisible. RetireSafe reports this as UNKNOWN instead of guessing.

14. Uncertainty and confidence intervals

Any number estimated from data is uncertain, especially when it is based on few events. A **95% confidence interval** gives a range that, by a standard statistical procedure, contains the true value in 95% of repeated experiments.

Wilson interval (for proportions)

For “k successes out of n” the naive interval $p \pm 1.96 \cdot \sqrt{p(1-p)/n}$ behaves badly when k is small (it can even go below zero). The **Wilson interval** fixes this:

$$\left[\frac{p + \frac{z^2}{2n} \pm z \cdot \sqrt{\frac{p(1-p)}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}} \right] \quad z = 1.96 \text{ for } 95\%$$

Example

TB1 found 5 reclaimable hostnames out of 16,000. $p = 0.031\%$. Wilson 95% interval = 0.013% to 0.073%. So we report “roughly 1 to 7 in 10,000”, not a falsely precise 0.03125%.

For rates, RetireSafe will use the *pessimistic* end of the range (a lower rate gives a longer, safer quarantine). That is how uncertainty makes the tool more careful rather than less.

15. Base rates: why we confirm before we alarm

When something is rare, even a good detector produces mostly false alarms. This is **Bayes' rule**:

$$P(\text{real} \mid \text{flagged}) = \text{sens} \cdot \pi / (\text{sens} \cdot \pi + \text{fpr} \cdot (1-\pi))$$

π = base rate (how common the real thing is)

sens = sensitivity (share of real cases the detector catches)

fpr = false-positive rate (share of harmless cases it wrongly flags)

Example

Our measured base rate is about $\pi = 0.0003$ (3 in 10,000). Take a decent detector with sens = 95% and fpr = 1%:

$P(\text{real} \mid \text{flagged}) = 0.95 \times 0.0003 / (0.95 \times 0.0003 + 0.01 \times 0.9997) \approx \mathbf{2.8\%}$. About 35 out of 36 alarms would be false!

Add a live confirmation probe that lowers fpr to 0.01%: the same formula gives $\approx \mathbf{74\%}$. This is why RetireSafe always confirms (e.g. checks “NoSuchBucket” live) before blocking.

16. Calibration: can we trust the probabilities?

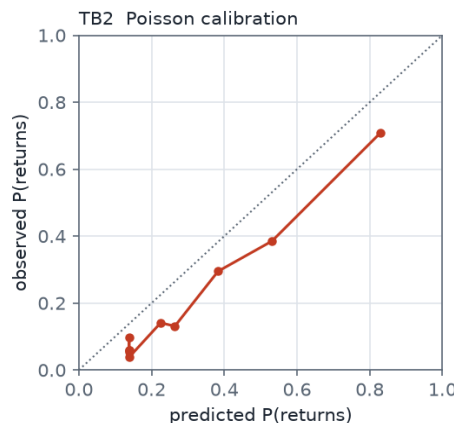
A model is **calibrated** if, among all the times it says “30% chance”, the event really happens about 30% of the time. A model can rank things correctly and still be badly calibrated.

How we test it: hold-out validation

Hide the end of the data. Fit the model on the beginning (days 0–24), make predictions for the hidden part (days 24–27.6), then compare with what really happened. Never test on the data you trained on.

Reliability diagram

Group predictions into bins (deciles), and plot the average predicted probability against the observed frequency. A perfectly calibrated model sits on the diagonal.



Our real result (TB2): every point lies below the diagonal, so the simple Poisson model predicted more returns than actually happened (1,161 vs 783, a factor of 1.48). The points still go up, so the model ranks resources well.

Scores

- **Brier score:** average of $(\text{predicted} - \text{actual})^2$, where actual is 1 or 0. Lower is better. Predicting 0.8 for something that happened costs $(0.8 - 1)^2 = 0.04$; predicting 0.8 for something that didn't costs 0.64.
- **Expected calibration error (ECE):** the average gap between the reliability points and the diagonal, weighted by bin size.
- **Isotonic regression:** a simple fix that learns a monotone mapping from the model's raw probability to an honest one using past data (proposed for RetireSafe).

17. Blast radius and the risk score

Blast radius = who would break if we deleted this: the number of distinct clients, the number of /24 networks, and the **external share** $e = \text{external clients} \div \text{all clients}$. A resource used by 3 internal machines is very different from one used by 15,000 strangers.

RetireSafe combines everything into one score that mirrors the five conditions. Because all five must hold, the probabilities are **multiplied**:

$$R = P(C1) \times P(C2) \times P(C3) \times P(C4) \times P(C5) \times \text{Impact}$$

If any factor is 0 (provably false), R is 0 and that path is cleared. If a factor is unknown, it is kept as a range rather than guessed, which widens the range of R. The score is used to sort findings so the most dangerous are fixed first. The score and the isotonic fix are *proposed*; the quarantine, the calibration check and the blast radius were run on real data.

18. Our three test beds in plain words

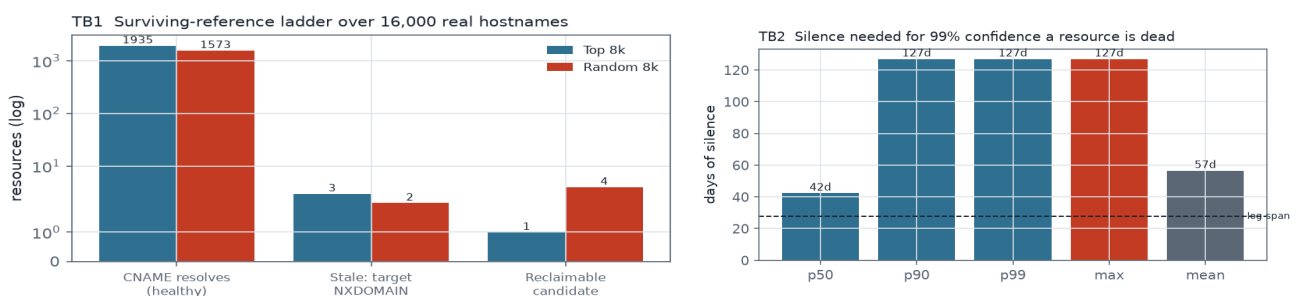
A **test bed** is a small experiment that tries an idea on real data before building the product. All three used real public data, made only read-only requests, and published no third-party hostnames.

TB1: How common are hijackable leftovers?

What we did: took 16,000 real hostnames from Cisco Umbrella's top-1M list (the 8,000 most popular plus 8,000 random ones), followed their CNAME chains, matched the targets against the fingerprint catalogue, and asked S3 whether matching buckets exist. **What we found: 5 reclaimable hostnames, 0.031%** (95% CI 0.013–0.073%): 4 pointing at S3 buckets that no longer exist, 1 at a missing Azure endpoint. Less-popular sites looked worse than the top sites (0.25% vs 0.05% of hostnames with an alias), though the intervals overlap. **So what:** the danger is rare and buried, which is why it needs an automated tool, and that tool must confirm findings (see §15).

TB2: How long is long enough?

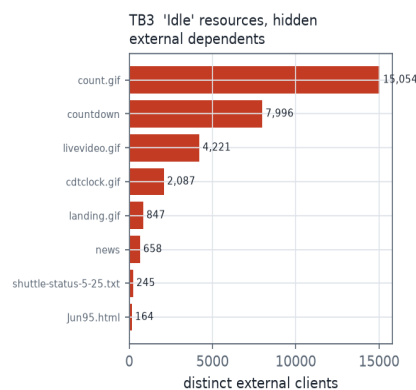
What we did: used NASA's real July 1995 web log (1.89 million requests, 7,133 resources) and computed each resource's quarantine D*. **What we found:** to be 99% sure, half the resources need **42 days** of silence and a third need **127 days**; 33% were requested only once all month. The Poisson model ranked resources well but over-predicted returns by 1.48×. **So what:** no single idle timeout is safe; use per-resource quarantines, recalibrate, and say UNKNOWN when the log is too short.



Left (TB1): almost every hostname resolves normally; only a handful reach the reclaimable rung (log scale). Right (TB2): days of silence needed for 99% confidence, by percentile.

TB3: How much hidden dependence is behind an “idle” resource?

What we did: looked at the NASA log the way a dashboard would, through the last 3 days only, and found 108 busy resources that looked dead. Then we counted everyone who had used them over the whole month. **What we found:** all 108 had outside users; /shuttle/countdown/count.gif had **0** recent hits but **15,054** distinct outside clients. (On a public site nearly everyone is external, so the striking part is the size, not the 100%.) **So what:** “looks unused to us” and “nobody depends on it” are very different; RetireSafe must look at history and outsiders, not just recent activity.



TB3: idle-looking resources and their hidden external users.

Planned but not yet run: an SPF dangling-include audit (TB4) and a scan of published software packages for S3 links (TB5).

19. The real-world incidents: cheat sheet

Case	When	What happened	Type of evidence
CDC Azure endpoint	Feb-Mar 2025	A CDC subdomain aliased to an abandoned Azure app was used to send visitors to scams and scareware	Observed malicious abuse (Infoblox)
Hazy Hawk campaign	Dec 2023 onward	French government Olympics site (DNS left after the event), an oercommons S3 bucket, subdomains under Deloitte, EY, PwC, UC Berkeley, UCL and others	Observed abuse, one campaign (Infoblox)
SubdoMailing	Disclosed Feb 2024	Expired domains still referenced by MSN and Swatch DNS/SPF; spoofed mail passed SPF	Observed campaign; 8,000+ domains is a vendor figure (Guardio)
NSDI 2024 study	Published 2024	20,904 hijacked cloud resources; about ⅓ lasted more than 65 days; 75% blackhat SEO	Peer-reviewed measurement
Abandoned S3 buckets	Feb 2025	About 150 buckets re-registered by researchers; 8M+ requests in 2 months	Researcher demonstration (watchTowr)
.mobi WHOIS	Sep 2024	Old WHOIS domain re-registered; 2.5M queries from 135,000+ systems; a CA accepted its answers for validation (stopped before a certificate was issued)	Researcher demonstration (watchTowr)

How to talk about the evidence honestly

Say	Don't say
Attackers have exploited forgotten references	Every stale reference is exploitable
A subdomain was hijacked	The organisation was breached
8 million requests reached reclaimed buckets	8 million infections
20,904 hijacked resources were found	20,904 organisations were attacked
Several examples belong to one campaign	These are many independent campaigns

Suggested pitch order (from the brief): CDC (a concrete victim) → NSDI (scale) → watchTowr S3 (why DNS-only checks miss application dependencies) → our test beds → live demo of a blocked-then-cleared deletion.

20. Ethics and responsible disclosure

- **Read-only research.** Only look things up (DNS queries, reading public responses). Never create, claim or register a resource you found dangling.
- **Don't publish targets.** Report aggregate counts. Keep any list of real vulnerable hostnames private.
- **Disclose responsibly.** Tell the owner privately, using their /.well-known/security.txt contact or bug-bounty programme, and give them time to fix it.
- **Demo only on what you own.** The hackathon demo uses the team's own sandbox account and test domain.
- **Be precise about claims.** Candidate $\neq$ confirmed; traffic $\neq$ compromise.

Watch out

Taking over someone else's subdomain "just to prove it", even harmlessly, can be illegal without permission. The research for this project never did.

21. Running the project code

```
git clone https://github.com/BOOMER16/RetireSafe.git && cd RetireSafe
pip install -r requirements.txt
./scripts/fetch_data.sh                # downloads the real datasets into ./data

cd testbeds
python3 tb1_dns_cname.py                # live DNS + S3 probe (~7 min for 16k names)
python3 tb2_traffic_survival.py         # quarantine + calibration (~1 min)
python3 tb3_blast_radius.py            # blast radius (~1 min)
python3 make_figs.py && python3 make_pdf.py

N_TOP=500 N_RAND=500 python3 tb1_dns_cname.py  # quick, smaller TB1 run
```

File	What it is
README.md	Overview and headline results
RetireSafe_Solution_Dossier.pdf	The 7-page research and solution report
docs/01 ... 07	Problem, test beds, architecture, maths, data, ethics, hackathon plan
testbeds/results/*.json	Exact numbers behind every claim
docs/RESEARCH_LOG.md	How the research was done, including corrections

22. FAQ: questions judges may ask

Isn't this just a subdomain-takeover scanner?

No. Scanners find problems after they exist. RetireSafe works *before* the deletion, across DNS, code, infrastructure and traffic, recognises provider protections, and produces a decision, a fix and an evidence record.

Why not just delete the DNS record too?

That fixes one reference. Code, package configs, SPF rules and outside consumers can still point at the name, and DNS is often managed by a different team. RetireSafe finds them all.

Why not keep every name forever?

That costs money and clutter. RetireSafe tells you which names really need retaining and for how long (the quarantine D*).

Doesn't AWS's new namespace solve it?

For new buckets in that namespace, yes. Legacy global buckets still exist and can't be renamed; other providers, DNS, SPF and code references remain. RetireSafe treats the namespace as evidence that clears condition 2.

How do you know a resource is unused?

We never “know”; we measure. Per-resource silence windows, calibrated probabilities, the number of outside users, and an explicit UNKNOWN when the logs are too short.

Your data is from 1995?

It's a large, real, public traffic log, which is rare. The method works on any access log, including a team's own CloudFront logs in the demo.

How accurate is TB1?

It's a 16,000-name sample of live infrastructure; it gives the order of magnitude (about 3 in 10,000), reported with a confidence interval.

Did you hack anything?

No. Only DNS lookups and public, read-only S3 queries. Nothing was claimed or registered, and no vulnerable hostnames were published.

23. Glossary

Term	Meaning
A / AAAA record	DNS record mapping a name to an IPv4 / IPv6 address.
Access log	A file where a web server writes one line per request.
Blast radius	Who would be affected if a resource disappeared or was hijacked.
Base rate (π)	How common something is in the population before any test.
Brier score	Average squared error of probability predictions; lower is better.
Calibration	Whether predicted probabilities match real frequencies.
CNAME	DNS alias: “this name is really that other name”.
CNAME chain	A series of aliases followed one after another.
Confidence interval	A range that expresses uncertainty around an estimate.
Consumer	A person or program that uses a resource or reference.
D* (quarantine)	Days of silence needed before believing a resource is unused, at a chosen confidence.
Dangling reference	A pointer to something you no longer control.
DKIM / DMARC	Email signature and email-policy standards that work alongside SPF.
DNS	Domain Name System: turns names into addresses.
eTLD+1 / registrable domain	The part of a name one owner can register (e.g. example.co.uk).
Evidence ladder	Stale reference → reclaimable endpoint → demonstrated impact.
Evidence record	RetireSafe's report of what was checked, how, and what couldn't be checked.
Fingerprint	The tell-tale sign that a provider resource is unclaimed (NXDOMAIN or an error page).
Global namespace	A pool of names shared by all customers of a provider; first come, first served.
Hold-out validation	Testing a model on data it was not trained on.
IaC	Infrastructure as Code: describing cloud resources in files (e.g. Terraform).
Isotonic regression	A method for correcting badly calibrated probabilities.
λ (lambda)	Rate: average events (requests) per unit of time.
NoSuchBucket	S3's error when a bucket name doesn't exist (so it may be claimable).
NXDOMAIN	DNS answer meaning “this name does not exist”.
Poisson process	Model of independent random events at a constant average rate.
prevent_destroy	Terraform setting that blocks plans deleting a resource.
Public Suffix List	List of shared suffixes like co.uk and github.io.
Reclaimable / reassignable	A released name that someone else can now claim.
Resolver	The DNS server that looks names up on your behalf.
S3 bucket	An AWS storage container with a name used in URLs.
SPF	DNS TXT record listing servers allowed to send email for a domain.
Subdomain takeover	An attacker controls content on your subdomain via a dangling record.
Test bed	A small real-data experiment run before building the product.
TTL	How long DNS answers may be cached.
Wilson interval	A confidence interval for proportions that works well with small counts.

24. Learning path and self-check

Read in this order (about 3 hours)

#	Read	Why
1	Microsoft Learn: Prevent dangling DNS entries and avoid subdomain takeover	Clear definition of the problem and Azure's defences
2	Infoblox (Mar 2025): the CDC incident	A concrete, recent victim
3	watchTowr (Feb 2025): 8 Million Requests Later	Why code and traffic matter, not just DNS
4	Guardio (Feb 2024): SubdoMailing	Email/SPF version of the problem
5	NSDI 2024, Friess et al.: Cloudy with a Chance of Cyberattacks	Measured scale (skim intro and results)
6	AWS: account regional namespaces; bucket owner condition	The newest protections
7	EdOverflow/can-i-take-over-xyz README	How each provider behaves
8	This repo: docs/02_testbeds.md and docs/04_mathematics.md	Our evidence and maths in detail

Links for all of these are in docs/SOURCES.md.

Can you explain these? (self-check)

- What a CNAME is, and what it means when its target returns NXDOMAIN.
- Why a deleted S3 bucket in the global namespace is dangerous, and what NoSuchBucket tells you.
- The five takeover conditions, and why breaking just one is enough.
- The difference between a stale reference, a reclaimable endpoint and demonstrated impact.
- How an SPF include: can hand email authority to an attacker.
- Why Terraform prevent_destroy is not enough on its own.
- What $e^{(-\lambda D)}$ means, and why rare resources need long quarantines.
- Why a rare problem plus an imperfect detector means mostly false alarms.
- What it means for a model to be calibrated, and what our reliability diagram showed.
- Our three headline numbers: 0.031% reclaimable; 42–127 days of quarantine; 15,054 hidden external users of an idle-looking file.
- The four verdicts, and why UNKNOWN is a feature.

You now know enough to explain RetireSafe to a judge, contribute to the build, and defend every number in the pitch.